Chapter 13

Graph neural networks

Chapter 10 described convolutional networks, which specialize in processing regular arrays of data (e.g., images). Chapter 12 described transformers, which specialize in processing sequences of variable length (e.g., text). This chapter describes *graph neural networks*. As the name suggests, these are neural architectures that process graphs (i.e., sets of nodes connected by edges).

There are three novel challenges associated with processing graphs. First, their topology is variable, and it is hard to design networks that are both sufficiently expressive and can cope with this variation. Second, graphs may be enormous; a graph representing connections between users of a social network might have a billion nodes. Third, there may only be a single monolithic graph available, so the usual protocol of training with many data examples and testing with new data is not always appropriate.

This chapter starts by presenting real-world examples of graphs. It then describes how to encode these graphs and how to formulate supervised learning problems for graphs. The algorithmic requirements for processing graphs are discussed, and these lead naturally to *graph convolutional networks*, a particular type of graph neural network.

13.1 What is a graph?

A graph is a very general structure and consists of a set of *nodes* or *vertices*, where pairs of nodes are connected by *edges* or *links*. Graphs are typically sparse; only a small subset of the possible edges are present.

Some objects in the real world naturally take the form of graphs. For example, road networks can be considered graphs where the nodes are physical locations, and the edges represent roads between them (figure 13.1a). Chemical molecules are small graphs where the nodes represent atoms, and the edges represent chemical bonds (figure 13.1b). Electrical circuits are graphs where the nodes represent components and junctions, and the edges are electrical connections (figure 13.1c).

Furthermore, many datasets can also be represented by graphs, even if this is not their obvious surface form. For example:

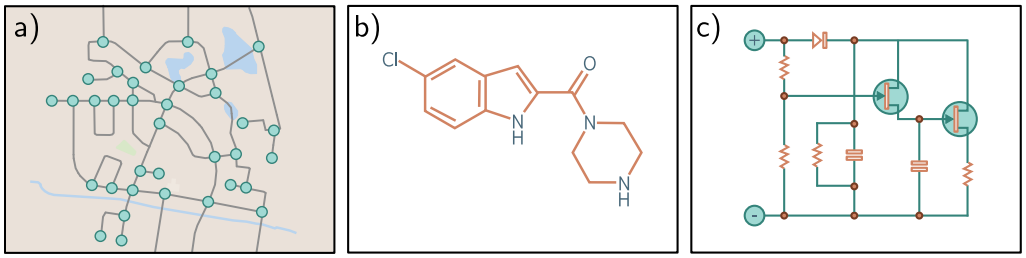

Figure 13.1 Real-world graphs. Some objects, such as a) road networks, b) molecules, and c) electrical circuits, are naturally structured as graphs.

- Social networks are graphs where nodes are people, and the edges represent friendships between them.
- The scientific literature can be viewed as a graph where the nodes are papers, and the edges represent citations.
- Wikipedia can be considered a graph where the nodes are articles, and the edges represent hyperlinks between articles.
- Computer programs can be represented as graphs where the nodes are syntax tokens (variables at different points in the program flow), and the edges represent computations involving these variables.
- Geometric point clouds can be represented as graphs. Here, each point is a node with edges connecting to other nearby points.
- Protein interactions in a cell can be expressed as graphs, where the nodes are the proteins, and there is an edge between two proteins if they interact.

In addition, a set (an unordered list) can be treated as a graph in which every member is a node and connects to every other. An image can be treated as a graph with regular topology, in which each pixel is a node with edges to the adjacent pixels.

13.1.1 Types of graphs

Graphs can be categorized in various ways. The social network in figure 13.2a contains *undirected edges*; each pair of individuals with a connection between them have mutually agreed to be friends, so there is no sense that the relationship is directional. In contrast, the citation network in figure 13.2b contains *directed edges*. Each paper cites other papers, and this relationship is inherently one-way.

Figure 13.2c depicts a *knowledge graph* that encodes a set of facts about objects by defining relations between them. Technically, this is a *directed heterogeneous multigraph*. It is heterogeneous because the nodes can represent different types of entities (e.g., people, countries, companies). It is a multigraph because there can be multiple edges of different types between any two nodes.

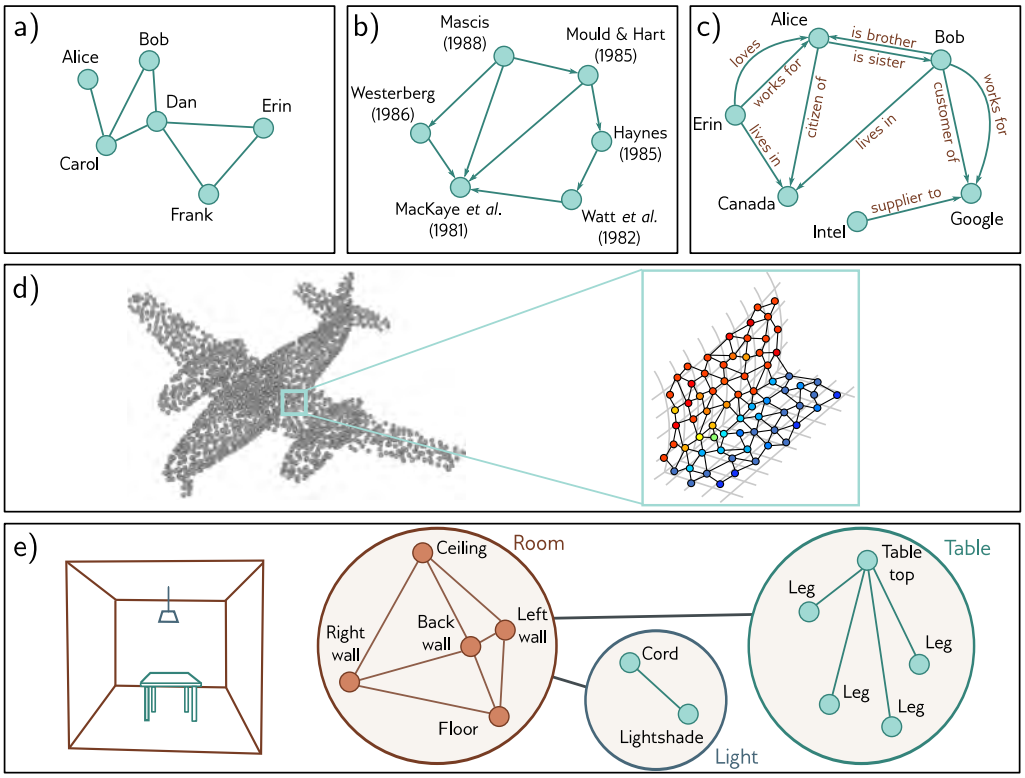

Figure 13.2 Types of graphs. a) A social network is an undirected graph; the connections between people are symmetric. b) A citation network is a directed graph; one publication cites another, so the relationship is asymmetric. c) A knowledge graph is a directed heterogeneous multigraph. The nodes are heterogeneous in that they represent different object types (people, places, companies) and multiple edges may represent different relations between each node. d) A point set can be converted to a graph by forming edges between nearby points. Each node has an associated position in 3D space, and this is termed a geometric graph (adapted from Hu et al., 2022). e) The scene on the left can be represented by a hierarchical graph. The topology of the room, table, and light are all represented by graphs. These graphs form nodes in a larger graph representing object adjacency (adapted from Fernández-Madrigal & González, 2002).

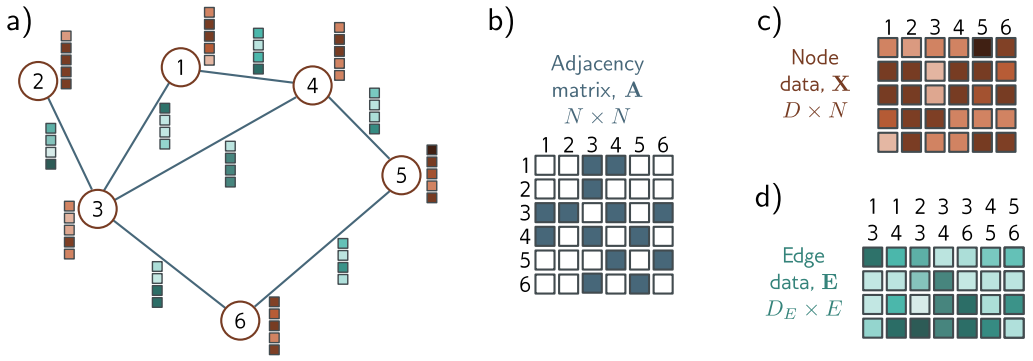

Figure 13.3 Graph representation. a) Example graph with six nodes and seven edges. Each node has an associated embedding of length five (brown vectors). Each edge has an associated embedding of length four (blue vectors). This graph can be represented by three matrices. b) The adjacency matrix is a binary matrix where element (m, n) is set to one if node m connects to node n. c) The node data matrix $\mathbf{X}$ contains the concatenated node embeddings. d) The edge data matrix $\mathbf{E}$ contains the edge embeddings.

The point set representing the airplane in figure 13.2d can be converted into a graph by connecting each point to its K nearest neighbors. The result is a *geometric graph* where each point is associated with a position in 3D space. Figure 13.2e represents a *hierarchical graph*. The table, light, and room are each described by graphs representing the adjacency of their respective components. These three graphs are themselves nodes in another graph that represents the topology of the objects in a larger model.

All types of graphs can be processed using deep learning. However, this chapter focuses on undirected graphs like the social network in figure 13.2a.

13.2 Graph representation

In addition to the graph structure itself, information is typically associated with each node. For example, in a social network, each individual might be characterized by a fixed-length vector representing their interests. Sometimes, the edges also have information attached. For example, in the road network example, each edge might be characterized by its length, number of lanes, frequency of accidents, and speed limit. The information at a node is stored in a *node embedding*, and the information at an edge is stored in an *edge embedding*.

More formally, a graph consists of a set of N nodes connected by a set of E edges. The graph can be encoded by three matrices $\mathbf{A}$, $\mathbf{X}$, and $\mathbf{E}$, representing the graph structure, node embeddings, and edge embeddings, respectively (figure 13.3).

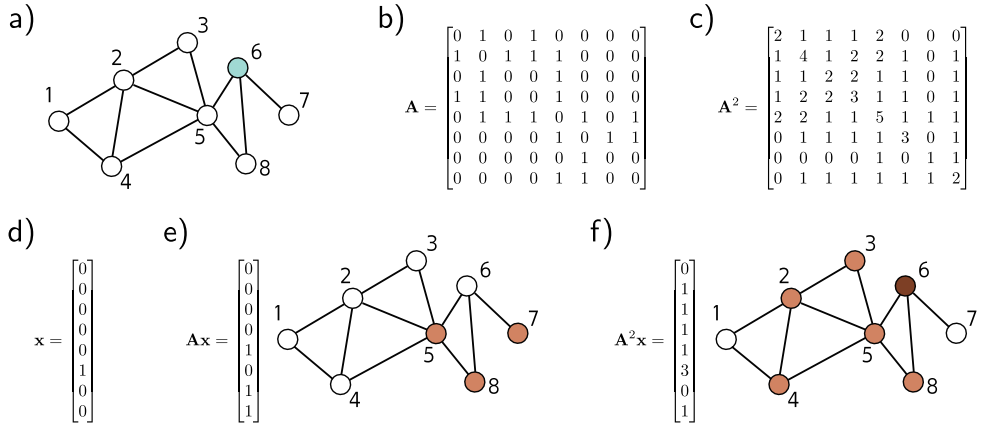

Figure 13.4 Properties of the adjacency matrix. a) Example graph. b) Position (m, n) of the adjacency matrix $\mathbf{A}$ contains the number of walks of length one from node m to node n. c) Position (m, n) of the squared adjacency matrix $\mathbf{A}^2$ contains the number of walks of length two from node m to node n. d) One hot vector representing node six, which was highlighted in panel (a). e) When we pre-multiply this vector by $\mathbf{A}$, the result contains the number of walks of length one from node six to each node; we can reach nodes five, seven, and eight in one move. f) When we pre-multiply this vector by $\mathbf{A}^2$, the resulting vector contains the number of walks of length two from node six to each node; we can reach nodes two, three, four, five, and eight in two moves, and we can return to the original node in three different ways (via nodes five, seven, and eight).

The graph structure is represented by the *adjacency matrix*, $\mathbf{A}$. This is an $N \times N$ matrix where entry (m, n) is set to one if there is an edge between nodes m and n and zero otherwise. For undirected graphs, this matrix is always symmetric. For large sparse graphs, it can be stored as a list of connections (m, n) to save memory.

Problems 13.1–13.2

The n^{th} node has an associated node embedding $\mathbf{x}^{(n)}$ of length D. These embeddings are concatenated and stored in the $D \times N$ node data matrix $\mathbf{X}$. Similarly, the e^{th} edge has an associated edge embedding $\mathbf{e}^{(e)}$ of length D_E. These edge embeddings are collected into the $D_E \times E$ matrix $\mathbf{E}$. For simplicity, we initially consider graphs that only have node embeddings and return to edge embeddings in section 13.9.

13.2.1 Properties of the adjacency matrix

The adjacency matrix can be used to find the neighbors of a node using linear algebra. Consider encoding the n^{th} node's position as a one-hot column vector (a vector with only one non-zero entry at position n, which is set to one). When we pre-multiply this vector by the adjacency matrix, it extracts the n^{th} column of the adjacency matrix and returns a vector with ones at the positions of the neighbors (i.e., all the places we can reach in

a walk of length one from the n^{th} node). If we repeat this procedure (i.e., pre-multiply by $\mathbf{A}$ again), the resulting vector contains the number of walks of length two from node n to every node (figures 13.4d–f).

Problems 13.3–13.4

In general, if we raise the adjacency matrix to the power of L, the entry at position (m, n) of $\mathbf{A}^L$ contains the number of unique *walks* of length L from node m to node n (figures 13.4a–c). This is not the same as the number of unique paths since the walks include routes that visit the same node more than once. Nonetheless, $\mathbf{A}^L$ still contains valuable information about the graph connectivity; a non-zero entry at position (m, n) indicates that the distance from m to n must be less than or equal to L.

Notebook 13.1 Encoding graphs

13.2.2 Permutation of node indices

Node indexing in graphs is arbitrary; permuting the node indices results in a permutation of the columns of the node data matrix $\mathbf{X}$ and a permutation of both the rows and columns of the adjacency matrix $\mathbf{A}$. However, the underlying graph is unchanged (figure 13.5). This is in contrast to images, where permuting the pixels creates a different image, and to text, where permuting the words creates a different sentence.

The operation of exchanging node indices can be expressed mathematically by a *permutation matrix*, $\mathbf{P}$. This is a matrix where exactly one entry in each row and column take the value one, and the remaining values are zero. When position (m, n) of the permutation matrix is set to one, it indicates that node m will become node n after the permutation. To map from one indexing to another, we use the operations:

Problem 13.5

$$\begin{aligned} \mathbf{X}' &= \mathbf{X}\mathbf{P} \\ \mathbf{A}' &= \mathbf{P}^T\mathbf{A}\mathbf{P}, \end{aligned} \tag{13.1}$$

where post-multiplying by $\mathbf{P}$ permutes the columns and pre-multiplying by $\mathbf{P}^T$ permutes the rows. It follows that any processing applied to the graph should also be indifferent to these permutations. Otherwise, the result will depend on the choice of node indices.

13.3 Graph neural networks, tasks, and loss functions

A graph neural network is a model that takes the node embeddings $\mathbf{X}$ and the adjacency matrix $\mathbf{A}$ as inputs and passes them through a series of K layers. The node embeddings are updated at each layer to create intermediate "hidden" representations $\mathbf{H}_k$ before finally computing output embeddings $\mathbf{H}_K$.

At the start of this network, each column of the input node embeddings $\mathbf{X}$ just contains information about the node itself. At the end, each column of the model output $\mathbf{H}_K$ includes information about the node and its context within the graph. This is similar to word embeddings passing through a transformer network. These represent words at the start, but represent the word meanings in the context of the sentence at the end.

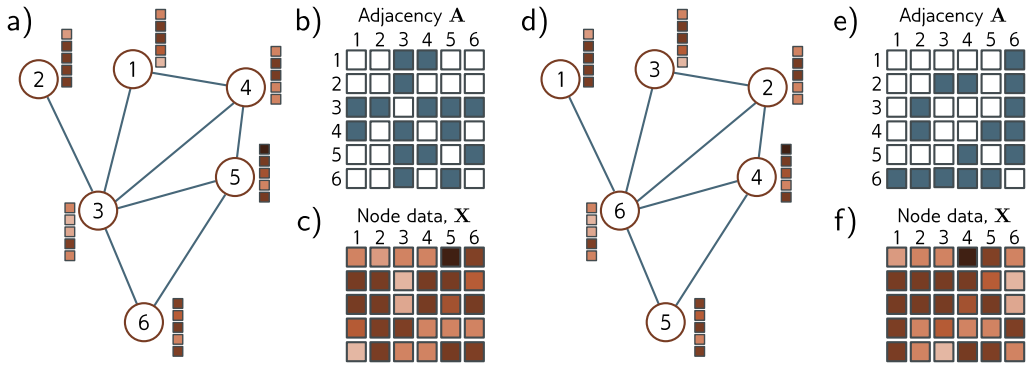

Figure 13.5 Permutation of node indices. a) Example graph, b) associated adjacency matrix and c) node embeddings. d) The same graph where the (arbitrary) order of the indices has been changed. e) The adjacency matrix and f) node matrix are now different. Consequently, any network layer that operates on the graph should be indifferent to the ordering of the nodes.

13.3.1 Tasks and loss functions

We defer discussion of graph neural network models until section 13.4 and first describe the types of problems these networks tackle and their associated loss functions. Supervised graph problems usually fall into one of three categories (figure 13.6).

Graph-level tasks: The network assigns a label or estimates one or more values from the entire graph, exploiting both the structure and node embeddings. For example, we might want to predict the temperature at which a molecule becomes liquid (a regression task) or whether a molecule is poisonous to human beings or not (a classification task).

For graph-level tasks, the output node embeddings are combined (e.g., by averaging), and the resulting vector is mapped via a linear transformation or neural network to a fixed-size vector. For regression, the mismatch between the result and the ground truth values is computed using the least squares loss. For binary classification, the output is passed through a sigmoid function, and the mismatch is calculated using the binary cross-entropy loss. Here, the probability that the graph belongs to class one might be given by:

$$Pr(y=1|\mathbf{X},\mathbf{A}) = \text{sig}\left[\beta_K + \boldsymbol{\omega}_K \mathbf{H}_K \mathbf{1}/N\right], \tag{13.2}$$

where the scalar β_K and $1 \times D$ vector $\boldsymbol{\omega}_K$ are learned parameters. Post-multiplying the output embedding matrix $\mathbf{H}_K$ by the column vector $\mathbf{1}$ that contains ones has the effect of summing together all the embeddings and subsequently dividing by the number of nodes N computes the average. This is known as *mean pooling* (see figure 10.11).

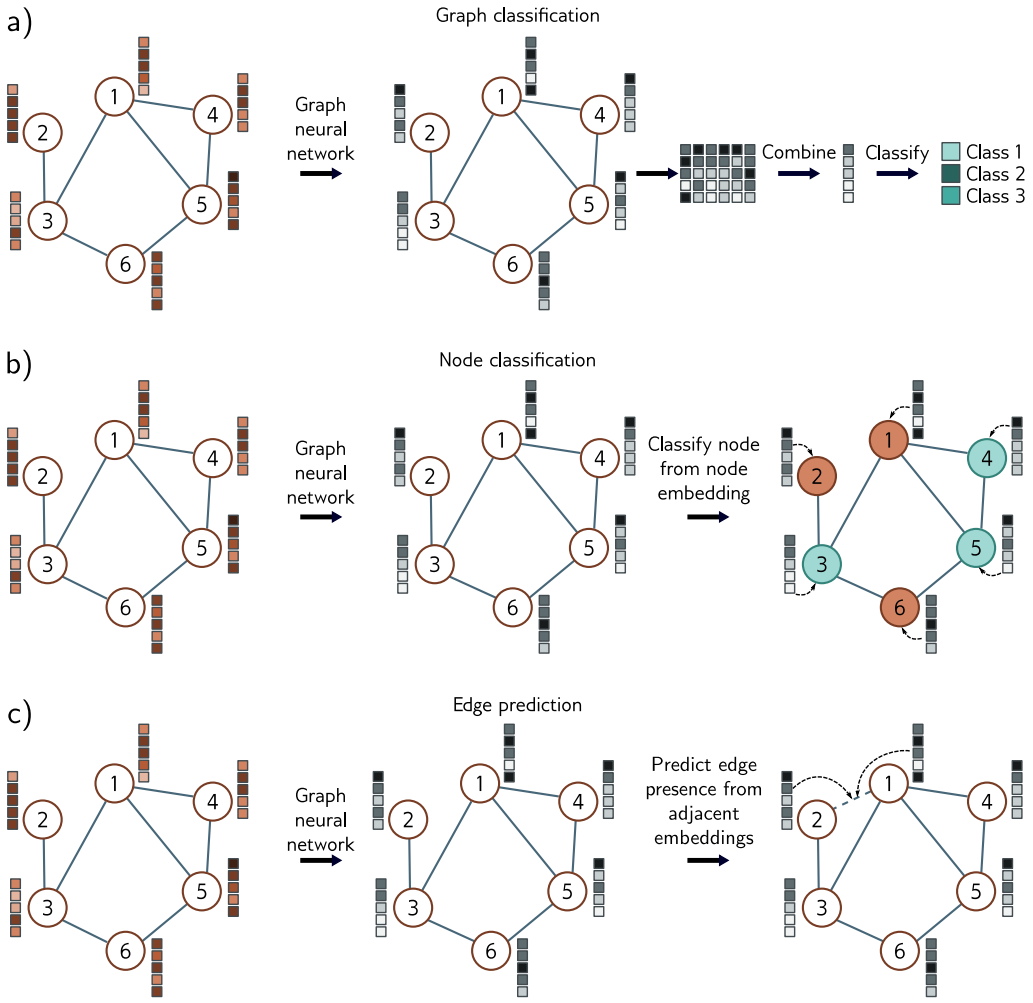

Figure 13.6 Common tasks for graphs. In each case, the input is a graph represented by its adjacency matrix and node embeddings. The graph neural network processes the node embeddings by passing them through a series of layers. The node embeddings at the last layer contain information about both the node and its context in the graph. a) Graph classification. The node embeddings are combined (e.g., by averaging) and then mapped to a fixed-size vector that is passed through a softmax function to produce class probabilities. b) Node classification. Each node embedding is used individually as the basis for classification (cyan and orange colors represent assigned node classes). c) Edge prediction. Node embeddings adjacent to the edge are combined (e.g., by taking the dot product) to compute a single number that is mapped via a sigmoid function to produce a probability that a missing edge should be present.

Node-level tasks: The network assigns a label (classification) or one or more values (regression) to each node of the graph, using both the graph structure and node embeddings. For example, given a graph constructed from a 3D point cloud similar to figure 13.2d, the goal might be to classify the nodes according to whether they belong to the wings or fuselage. Loss functions are defined in the same way as for graph-level tasks, except that now this is done independently at each node n:

$$Pr(y^{(n)} = 1|\mathbf{X}, \mathbf{A}) = \text{sig}\left[\beta_K + \boldsymbol{\omega}_K \mathbf{h}_K^{(n)}\right]. \tag{13.3}$$

Edge prediction tasks: The network predicts whether or not there should be an edge between nodes n and m. For example, in the social network setting, the network might predict whether two people know and like each other and suggest that they connect if that is the case. This is a binary classification task where the two node embeddings must be mapped to a single number representing the probability that the edge is present. One possibility is to take the dot product of the node embeddings and pass the result through a sigmoid function to create the probability:

$$Pr(y^{(mn)} = 1|\mathbf{X}, \mathbf{A}) = \text{sig}\left[\mathbf{h}^{(m)T}\mathbf{h}^{(n)}\right]. \tag{13.4}$$

13.4 Graph convolutional networks

There are many types of graph neural networks, but here we focus on *spatial-based convolutional graph neural networks*, or *GCNs* for short. These models are convolutional in that they update each node by aggregating information from nearby nodes. As such, they induce a *relational inductive bias* (i.e., a bias toward prioritizing information from neighbors). They are spatial-based because they use the original graph structure. This contrasts with *spectral-based methods*, which apply convolutions in the Fourier domain.

Each layer of the GCN is a function $\mathbf{F}[\bullet]$ with parameters $\boldsymbol{\Phi}$ that takes the node embeddings and adjacency matrix and outputs new node embeddings. The network can hence be written as:

$$\begin{aligned}
\mathbf{H}_1 &= \mathbf{F}[\mathbf{X}, \mathbf{A}, \boldsymbol{\phi}_0] \\
\mathbf{H}_2 &= \mathbf{F}[\mathbf{H}_1, \mathbf{A}, \boldsymbol{\phi}_1] \\
\mathbf{H}_3 &= \mathbf{F}[\mathbf{H}_2, \mathbf{A}, \boldsymbol{\phi}_2] \\
\vdots &= \vdots \\
\mathbf{H}_K &= \mathbf{F}[\mathbf{H}_{K-1}, \mathbf{A}, \boldsymbol{\phi}_{K-1}],
\end{aligned} \tag{13.5}$$

where $\mathbf{X}$ is the input, $\mathbf{A}$ is the adjacency matrix, $\mathbf{H}_k$ contains the modified node embeddings at the k^{th} layer, and $\boldsymbol{\phi}_k$ denotes the parameters that map from layer k to layer $k+1$.

13.4.1 Equivariance and invariance

We noted before that the indexing of the nodes in the graph is arbitrary, and any permutation of the node indices does not change the graph. It is hence imperative that any model respects this property. It follows that each layer must be equivariant (see section 10.1) with respect to permutations of the node indices. In other words, if we permute the node indices, the node embeddings at each stage will be permuted in the same way. In mathematical terms, if $\mathbf{P}$ is a permutation matrix, then we must have:

$$\mathbf{H}_{k+1}\mathbf{P} = \mathbf{F}[\mathbf{H}_k\mathbf{P}, \mathbf{P}^T\mathbf{A}\mathbf{P}, \boldsymbol{\phi}_k]. \tag{13.6}$$

For node classification and edge prediction tasks, the output should also be equivariant with respect to permutations of the node indices. However, for graph-level tasks, the final layer aggregates information from across the graph, so the output is invariant to the node order. In fact, the output layer from equation 13.2 achieves this because:

Problem 13.6

$$y = \text{sig}\left[\beta_K + \boldsymbol{\omega}_K\mathbf{H}_K\mathbf{1}/N\right] = \text{sig}\left[\beta_K + \boldsymbol{\omega}_K\mathbf{H}_K\mathbf{P}\mathbf{1}/N\right], \tag{13.7}$$

for any permutation matrix $\mathbf{P}$ (see problem 13.6).

This mirrors the case for images, where segmentation should be equivariant to geometric transformations, and image classification should be invariant (figure 10.1). Here, convolutional and pooling layers partially achieve this with respect to translations, but there is no known way to guarantee these properties exactly for more general transformations. However, for graphs, it is possible to define networks that ensure equivariance or invariance to permutations.

13.4.2 Parameter sharing

Chapter 10 argued applying fully connected networks to images isn't sensible because this requires the network to learn how to recognize an object separately at every image position. Instead, we used convolutional layers that processed every position in the image identically. This reduced the number of parameters and introduced an inductive bias that forced the model to treat every part of the image in the same way.

The same argument can be made about nodes in a graph. We could learn a model with separate parameters associated with each node. However, now the network must independently learn the meaning of the connections in the graph at each position, and training would require many graphs with the same topology. Instead, we build a model that uses the same parameters at every node, reducing the number of parameters and sharing what the network learns at each node across the entire graph.

Recall that a convolution (equation 10.3) updates a variable by taking a weighted sum of information from its neighbors. One way to think of this is that each neighbor sends a message to the variable of interest, which aggregates these messages to form the update. When we considered images, the neighbors were pixels from a fixed-size square region around the current position, so the spatial relationships at each position are the same. However, in a graph, each node may have a different number of neighbors, and there are no consistent relationships; there is no sense that we can weight information

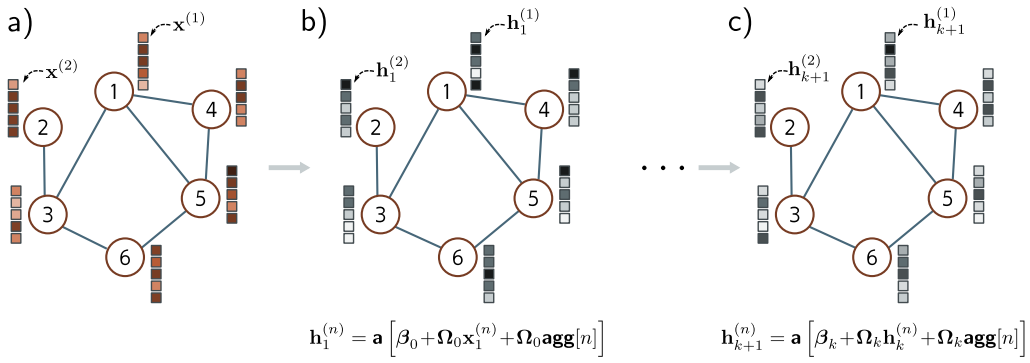

Figure 13.7 Simple Graph CNN layer. a) Input graph consists of structure (embodied in graph adjacency matrix $\mathbf{A}$, not shown) and node embeddings (stored in columns of $\mathbf{X}$). b) Each node in the first hidden layer is updated by (i) aggregating the neighboring nodes to form a single vector, (ii) applying a linear transformation $\boldsymbol{\Omega}_0$ to the aggregated vector, (iii) applying the same linear transformation $\boldsymbol{\Omega}_0$ to the original node, (iv) adding these together with a bias $\boldsymbol{\beta}_0$, and finally (v) applying a nonlinear activation function $\mathbf{a}[\bullet]$ like a ReLU. c) This process is repeated at subsequent layers (but with different parameters for each layer) until we produce the final embeddings at the end of the network.

from a node that is "above" the node of interest differently to information from a node that is "below" it.

13.4.3 Example GCN layer

These considerations lead to a simple GCN layer (figure 13.7). At each node n in layer k, we aggregate information from neighboring nodes by summing their node embeddings $\mathbf{h}_\bullet$:

$$\mathbf{agg}[n,k] = \sum_{m \in \mathrm{ne}[n]} \mathbf{h}_k^{(m)}, \tag{13.8}$$

where $\mathrm{ne}[n]$ returns the set of indices of the neighbors of node n. Then we apply a linear transformation $\boldsymbol{\Omega}_k$ to the embedding $\mathbf{h}_k^{(n)}$ at the current node and to this aggregated value, add a bias term $\boldsymbol{\beta}_k$, and pass the result through a nonlinear activation function $\mathbf{a}[\bullet]$, which is applied independently to every member of its vector argument:

$$\mathbf{h}_{k+1}^{(n)} = \mathbf{a}\left[\boldsymbol{\beta}_k + \boldsymbol{\Omega}_k \cdot \mathbf{h}_k^{(n)} + \boldsymbol{\Omega}_k \cdot \mathbf{agg}[n,k]\right]. \tag{13.9}$$

We can write this more succinctly by noting that post-multiplication of a matrix by a vector returns a weighted sum of its columns. The n^{th} column of the adjacency matrix $\mathbf{A}$ contains ones at the positions of the neighbors. Hence, if we collect the node

embeddings into the $D \times N$ matrix $\mathbf{H}_k$ and post-multiply by the adjacency matrix $\mathbf{A}$, the n^{th} column of the result is $\mathbf{agg}[n, k]$. The update for the nodes is now:

$$\begin{aligned}\mathbf{H}_{k+1} &= \mathbf{a}\left[\boldsymbol{\beta}_k \mathbf{1}^T + \boldsymbol{\Omega}_k \mathbf{H}_k + \boldsymbol{\Omega}_k \mathbf{H}_k \mathbf{A}\right] \\ &= \mathbf{a}\left[\boldsymbol{\beta}_k \mathbf{1}^T + \boldsymbol{\Omega}_k \mathbf{H}_k (\mathbf{A} + \mathbf{I})\right], \end{aligned} \tag{13.10}$$

where $\mathbf{1}$ is an $N \times 1$ vector containing ones. Here, the nonlinear activation function $\mathbf{a}[\bullet]$ is applied independently to every member of its matrix argument.

This layer satisfies the design considerations: it is equivariant to permutations of the node indices, can cope with any number of neighbors, exploits the graph structure to provide a relational inductive bias, and shares parameters throughout the graph.

Problem 13.7

13.5 Example: graph classification

We now combine these ideas to describe a network that classifies molecules as toxic or harmless. The network inputs are the adjacency matrix and node embedding matrix $\mathbf{X}$. The adjacency matrix $\mathbf{A} \in \mathbb{R}^{N \times N}$ derives from the molecular structure. The columns of the node embedding matrix $\mathbf{X} \in \mathbb{R}^{118 \times N}$ are one-hot vectors indicating which of the 118 elements of the periodic table are present. In other words, they are vectors of length 118 where every position is zero except for the position corresponding to the relevant element, which is set to one. The node embeddings can be transformed to an arbitrary size D by the first weight matrix $\boldsymbol{\Omega}_0 \in \mathbb{R}^{D \times 118}$.

Notebook 13.2
Graph classification

The network equations are:

$$\begin{aligned}\mathbf{H}_1 &= \mathbf{a}\left[\boldsymbol{\beta}_0 \mathbf{1}^T + \boldsymbol{\Omega}_0 \mathbf{X}(\mathbf{A} + \mathbf{I})\right] \\ \mathbf{H}_2 &= \mathbf{a}\left[\boldsymbol{\beta}_1 \mathbf{1}^T + \boldsymbol{\Omega}_1 \mathbf{H}_1 (\mathbf{A} + \mathbf{I})\right] \\ \vdots &= \vdots \\ \mathbf{H}_K &= \mathbf{a}\left[\boldsymbol{\beta}_{K-1} \mathbf{1}^T + \boldsymbol{\Omega}_{K-1} \mathbf{H}_{k-1} (\mathbf{A} + \mathbf{I})\right] \\ \mathrm{f}[\mathbf{X}, \mathbf{A}, \boldsymbol{\Phi}] &= \mathrm{sig}\left[\beta_K + \boldsymbol{\omega}_K \mathbf{H}_K \mathbf{1}/N\right], \end{aligned} \tag{13.11}$$

where the network output $\mathrm{f}[\mathbf{X}, \mathbf{A}, \boldsymbol{\Phi}]$ is a single value that determines the probability that the molecule is toxic (see equation 13.2).

13.5.1 Training with batches

Given I training graphs $\{\mathbf{X}_i, \mathbf{A}_i\}$ and their labels y_i, the parameters $\boldsymbol{\Phi} = \{\boldsymbol{\beta}_k, \boldsymbol{\Omega}_k\}_{k=0}^K$ can be learned using SGD and the binary cross-entropy loss (equation 5.19). Fully connected networks, convolutional networks, and transformers all exploit the parallelism of modern hardware to process an entire batch of training examples concurrently. To this end, the batch elements are concatenated into a higher-dimensional tensor (section 7.4.2).

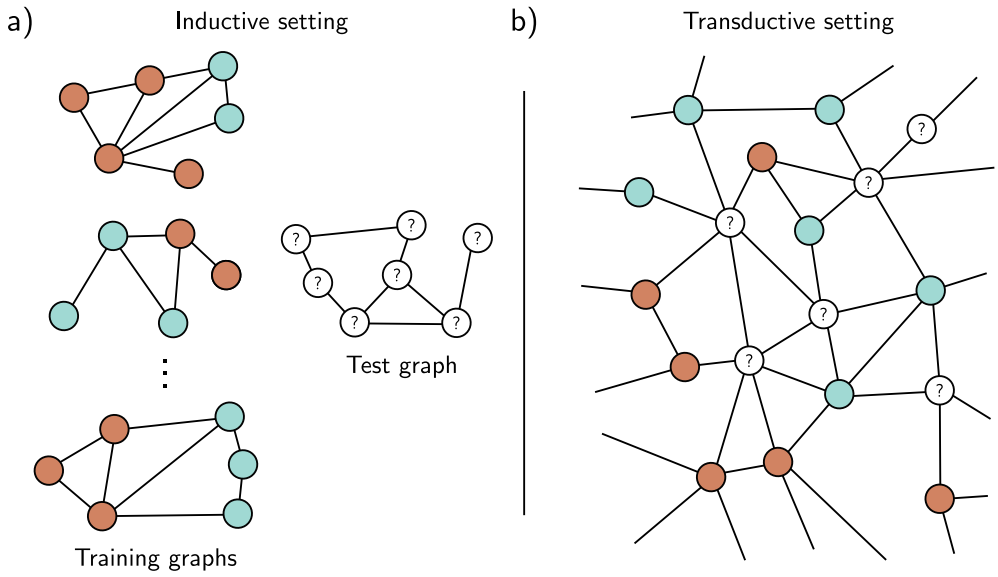

Figure 13.8 Inductive vs. transductive problems. a) Node classification task in the inductive setting. We are given a set of I training graphs, where the node labels (orange and cyan colors) are known. After training, we are given a test graph and must assign labels to each node. b) Node classification in the transductive setting. There is one large graph in which some nodes have labels (orange and cyan colors), and others are unknown. We train the model to predict the known labels correctly and then examine the predictions at the unknown nodes.

However, each graph may have a different number of nodes. Hence, the matrices $\mathbf{X}_i$ and $\mathbf{A}_i$ have different sizes, and there is no way to concatenate them into 3D tensors.

Luckily, a simple trick allows us to process the whole batch in parallel. The graphs in the batch are treated as disjoint components of a single large graph. The network can then be run as a single instance of the network equations. The mean pooling is carried out only over the individual graphs to make a single representation per graph that can be fed into the loss function.

13.6 Inductive vs. transductive models

Until this point, all of the models in this book have been *inductive*: we exploit a training set of labeled data to learn the relation between the inputs and outputs. Then we apply this to new test data. One way to think of this is that we are learning the rule that maps inputs to outputs and then applying it elsewhere.

By contrast, a *transductive* model considers both the labeled and unlabeled data

at the same time. It does not produce a rule but merely a labeling for the unknown outputs. This is sometimes termed *semi-supervised learning*. It has the advantage that it can use patterns in the unlabeled data to help make its decisions. However, it has the disadvantage that the model needs to be retrained when extra unlabeled data are added.

Both problem types are commonly encountered for graphs (figure 13.8). Sometimes, we have many labeled graphs and learn a mapping between the graph and the labels. For example, we might have many molecules, each labeled according to whether it is toxic to humans. We learn the rule that maps the graph to the toxic/non-toxic label and then apply this rule to new molecules. However, sometimes there is a single monolithic graph. In the graph of scientific paper citations, we might have labels indicating the field (physics, biology, etc.) for some nodes and wish to label the remaining nodes. Here, the training and test data are irrevocably connected.

Graph-level tasks only occur in the inductive setting where there are training and test graphs. However, node-level tasks and edge prediction tasks can occur in either setting. In the transductive case, the loss function minimizes the mismatch between the model output and the ground truth where this is known. New predictions are computed by running the forward pass and retrieving the results where the ground truth is unknown.

13.7 Example: node classification

As a second example, consider a binary node classification task in a transductive setting. We start with a commercial-sized graph with millions of nodes. Some nodes have ground truth binary labels, and the goal is to label the remaining unlabeled nodes. The body of the network will be the same as in the previous example (equation 13.11) but with a different final layer that produces an output vector of size $1 \times N$:

$$\mathbf{f}[\mathbf{X}, \mathbf{A}, \boldsymbol{\Phi}] = \mathbf{sig}\left[\beta_K \mathbf{1}^T + \boldsymbol{\omega}_K \mathbf{H}_K\right], \tag{13.12}$$

where the function $\mathbf{sig}[\bullet]$ applies the sigmoid function independently to every element of the row vector input. As usual, we use the binary cross-entropy loss, but now only at nodes where we know the ground truth label y. Note that equation 13.12 is just a vectorized version of the node classification loss from equation 13.3.

Training this network raises two problems. First, it is logistically difficult to train a graph neural network of this size. Consider that we must store the node embeddings at every network layer in the forward pass. This will involve both storing and processing a structure several times the size of the entire graph, and this may not be practical. Second, we have only a single graph, so it's not obvious how to perform stochastic gradient descent. How can we form a batch if there is only a single object?

13.7.1 Choosing batches

One way to form a batch is to choose a random subset of labeled nodes at each training step. Each node depends on its neighbors in the previous layer. These, in turn, depend

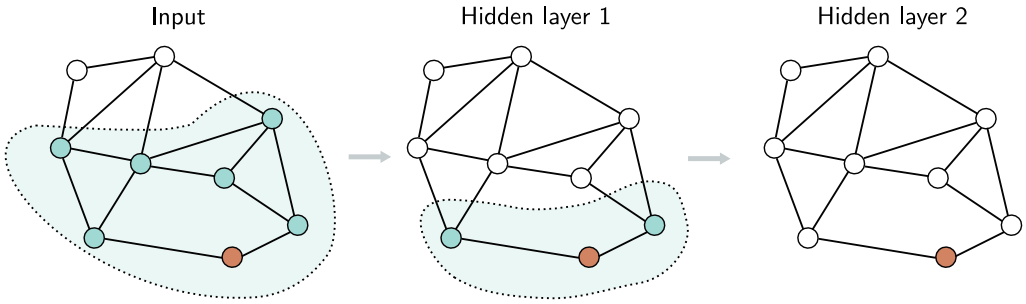

Figure 13.9 Receptive fields in graph neural networks. Consider the orange node in hidden layer two (right). This receives input from the nodes in the 1-hop neighborhood in hidden layer one (shaded region in center). These nodes in hidden layer one receive inputs from their neighbors in turn, and the orange node in layer two receives inputs from all the input nodes in the 2-hop neighborhood (shaded area on left). The region of the graph that contributes to a given node is equivalent to the notion of a receptive field in convolutional neural networks.

on their neighbors in the layer before, so (similarly to convolutional networks) each node has a receptive field (figure 13.9). The receptive field region is termed the *k-hop neighborhood*. We can hence perform a gradient descent step using the graph that forms the union of the k-hop neighborhoods of the batch nodes; the remaining inputs do not contribute.

Unfortunately, if there are many layers and the graph is densely connected, every input node may be in the receptive field of every output, and this may not reduce the graph size at all. This is known as the *graph expansion problem*. Two approaches that tackle this problem are *neighborhood sampling* and *graph partitioning*.

Neighborhood sampling: The full graph that feeds into the batch of nodes is sampled, thereby reducing the connections at each network layer (figure 13.10). For example, we might start with the batch nodes and randomly sample a fixed number of their neighbors in the previous layer. Then, we randomly sample a fixed number of *their* neighbors in the layer before, and so on. The graph still increases in size with each layer but in a much more controlled way. This is done anew for each batch, so the contributing neighbors differ even if the same batch is drawn twice. This is also reminiscent of dropout (section 9.3.3) and adds some regularization.

Notebook 13.3 Neighborhood sampling

Graph partitioning: A second approach is to cluster the original graph into disjoint subsets of nodes (i.e., smaller graphs that are not connected to one another) before processing (figure 13.11). There are standard algorithms to choose these subsets to maximize the number of internal links. These smaller graphs can each be treated as batches, or a random subset of them can be combined to form a batch (reinstating any edges between them from the original graph).

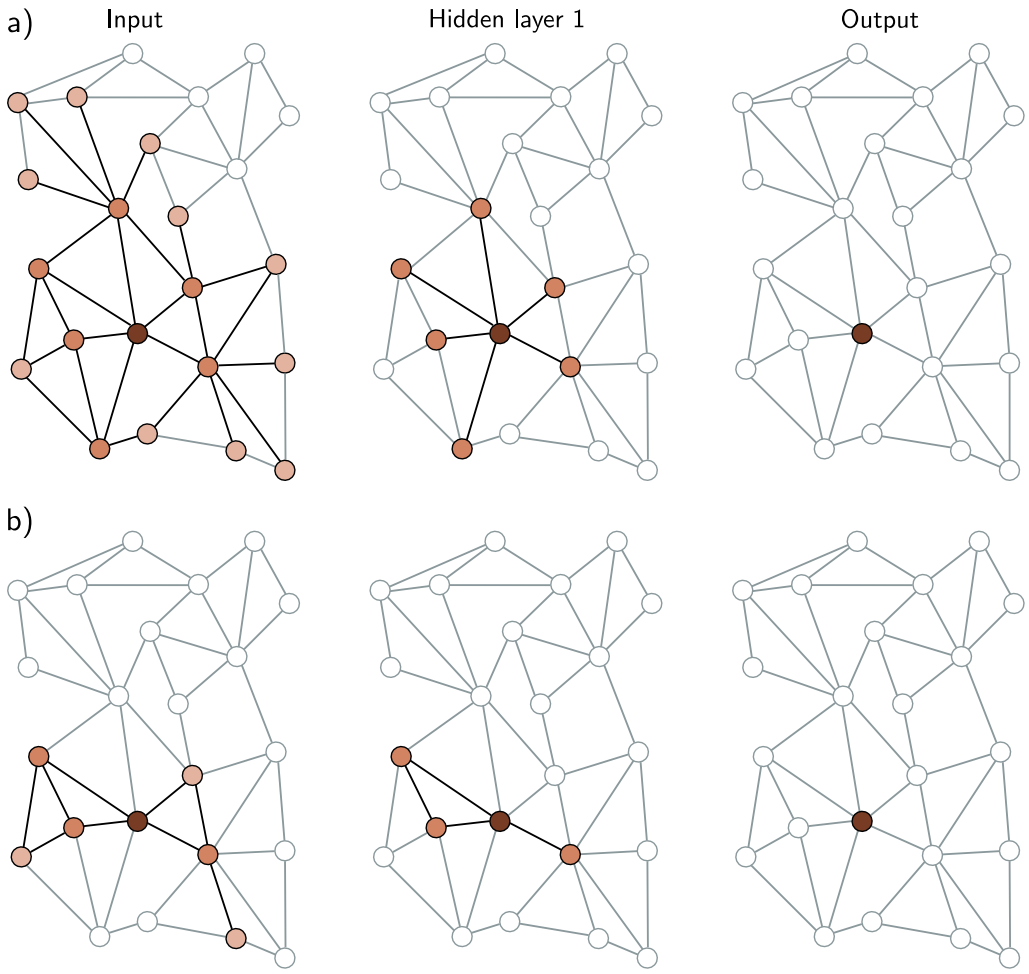

Figure 13.10 Neighborhood sampling. a) One way of forming batches on large graphs is to choose a subset of labeled nodes in the output layer (here, just one node in layer two, right) and then working back to find all of the nodes in the K-hop neighborhood (receptive field). Only this sub-graph is needed to train this batch. Unfortunately, if the graph is densely connected, this may retain a large proportion of the graph. b) One solution is neighborhood sampling. As we work back from the final layer, we select a subset of neighbors (here, three) in the layer before and a subset of the neighbors of these in the layer before that. This restricts the size of the graph for training the batch. In all panels, the brightness represents the distance from the original node.

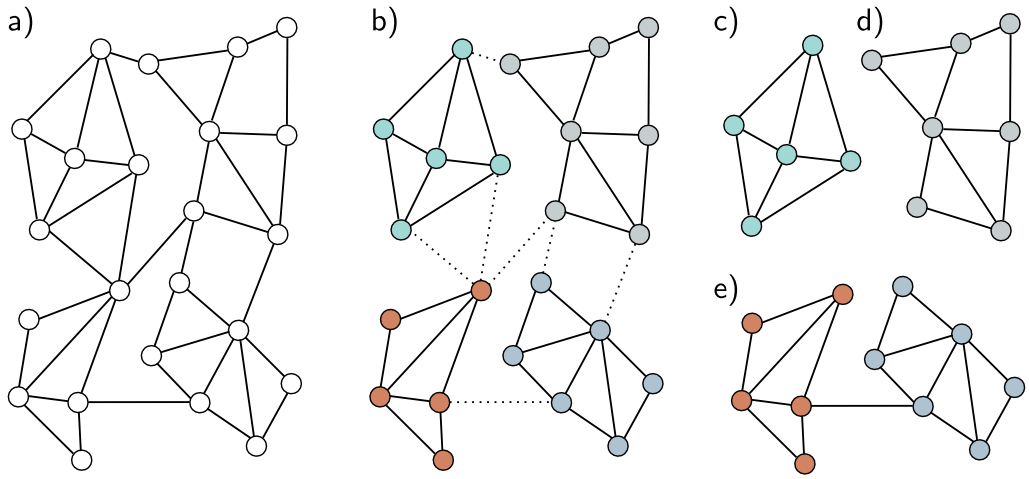

Figure 13.11 Graph partitioning. a) Input graph. b) The input graph is partitioned into smaller subgraphs using a principled method that removes the fewest edges. c-d) We can now use these subgraphs as batches to train in a transductive setting, so here, there are four possible batches. e) Alternatively, we can use combinations of the subgraphs as batches, reinstating the edges between them. If we use pairs of subgraphs, there would be six possible batches here.

Given one of the above methods to form batches, we can now train the network parameters in the same way as for the inductive setting, dividing the labeled nodes into train, test, and validation sets as desired; we have effectively converted a transductive problem to an inductive one. To perform inference, we compute predictions for the unknown nodes based on their k-hop neighborhood. Unlike training, this does not require storing the intermediate representations, so it is much more memory efficient.

13.8 Layers for graph convolutional networks

In the previous examples, we combined messages from adjacent nodes by summing them together with the transformed current node. This was accomplished by post-multiplying the node embedding matrix $\mathbf{H}$ by the adjacency matrix plus the identity $\mathbf{A}+\mathbf{I}$. We now consider different approaches to both (i) the combination of the current embedding with the aggregated neighbors and (ii) the aggregation process itself.

13.8.1 Combining current node and aggregated neighbors

In the example GCN layer above, we combined the aggregated neighbors $\mathbf{HA}$ with the current nodes $\mathbf{H}$ by just summing them:

$$\mathbf{H}_{k+1} = \mathbf{a}\Big[\boldsymbol{\beta}_k\mathbf{1}^T + \boldsymbol{\Omega}_k\mathbf{H}_k(\mathbf{A}+\mathbf{I})\Big]. \tag{13.13}$$

In another variation, the current node is multiplied by a factor of $(1+\epsilon_k)$ before contributing to the sum, where ϵ_k is a learned scalar that is different for each layer:

$$\mathbf{H}_{k+1} = \mathbf{a}\Big[\boldsymbol{\beta}_k\mathbf{1}^T + \boldsymbol{\Omega}_k\mathbf{H}_k(\mathbf{A}+(1+\epsilon_k)\mathbf{I})\Big]. \tag{13.14}$$

This is known as *diagonal enhancement*. A related variation applies a different linear transform $\boldsymbol{\Psi}_k$ to the current node:

$$\begin{aligned}\mathbf{H}_{k+1} &= \mathbf{a}\left[\boldsymbol{\beta}_k\mathbf{1}^T + \boldsymbol{\Omega}_k\mathbf{H}_k\mathbf{A} + \boldsymbol{\Psi}_k\mathbf{H}_k\right] \\ &= \mathbf{a}\left[\boldsymbol{\beta}_k\mathbf{1}^T + \begin{bmatrix}\boldsymbol{\Omega}_k & \boldsymbol{\Psi}_k\end{bmatrix}\begin{bmatrix}\mathbf{H}_k\mathbf{A} \\ \mathbf{H}_k\end{bmatrix}\right] \\ &= \mathbf{a}\left[\boldsymbol{\beta}_k\mathbf{1}^T + \boldsymbol{\Omega}'_k\begin{bmatrix}\mathbf{H}_k\mathbf{A} \\ \mathbf{H}_k\end{bmatrix}\right],\end{aligned} \tag{13.15}$$

where we have defined $\boldsymbol{\Omega}'_k = \begin{bmatrix}\boldsymbol{\Omega}_k & \boldsymbol{\Psi}_k\end{bmatrix}$ in the third line.

13.8.2 Residual connections

With residual connections, the aggregated representation from the neighbors is transformed and passed through the activation function before summation or concatenation with the current node. For the latter case, the associated network equations are:

$$\mathbf{H}_{k+1} = \begin{bmatrix}\mathbf{a}\left[\boldsymbol{\beta}_k\mathbf{1}^T + \boldsymbol{\Omega}_k\mathbf{H}_k\mathbf{A}\right] \\ \mathbf{H}_k\end{bmatrix}. \tag{13.16}$$

13.8.3 Mean aggregation

The above methods aggregate the neighbors by summing the node embeddings. However, it's possible to combine the embeddings in different ways. Sometimes it's better to take the average of the neighbors rather than the sum; this can be superior if the embedding information is more important and the structural information less so since the magnitude of the neighborhood contributions will not depend on the number of neighbors:

$$\mathbf{agg}[n] = \frac{1}{|\mathrm{ne}[n]|}\sum_{m\in\mathrm{ne}[n]}\mathbf{h}_m, \tag{13.17}$$

where as before, ne[n] denotes a set containing the indices of the neighbors of the n^{th} node. Equation 13.17 can be computed neatly in matrix form by introducing the diagonal $N \times N$ degree matrix $\mathbf{D}$. Each non-zero element of this matrix contains the number of neighbors for the associated node. It follows that each diagonal element in the inverse matrix $\mathbf{D}^{-1}$ contains the denominator that we need to compute the average. The new GCN layer can be written as:

Problem 13.8

$$\mathbf{H}_{k+1} = \mathbf{a}\Big[\boldsymbol{\beta}_k \mathbf{1}^T + \boldsymbol{\Omega}_k \mathbf{H}_k (\mathbf{A}\mathbf{D}^{-1} + \mathbf{I})\Big]. \tag{13.18}$$

13.8.4 Kipf normalization

There are many variations of graph neural networks based on mean aggregation. Sometimes the current node is included with its neighbors in the mean computation rather than treated separately. In Kipf normalization, the sum of the node representations is normalized as:

Problem 13.9

$$\mathbf{agg}[n] = \sum_{m \in \text{ne}[n]} \frac{\mathbf{h}_m}{\sqrt{|\text{ne}[n]||\text{ne}[m]|}}, \tag{13.19}$$

with the logic that information coming from nodes with a very large number of neighbors should be down-weighted since there are many connections and they provide less unique information. This can also be expressed in matrix form using the degree matrix:

$$\mathbf{H}_{k+1} = \mathbf{a}\Big[\boldsymbol{\beta}_k \mathbf{1}^T + \boldsymbol{\Omega}_k \mathbf{H}_k (\mathbf{D}^{-1/2}\mathbf{A}\mathbf{D}^{-1/2} + \mathbf{I})\Big]. \tag{13.20}$$

13.8.5 Max pooling aggregation

An alternative operation that is also invariant to permutation is computing the maximum of a set of objects. The *max pooling* aggregation operator is:

$$\mathbf{agg}[n] = \max_{m \in \text{ne}[n]} \big[\mathbf{h}_m\big], \tag{13.21}$$

where the operator $\mathbf{max}[\bullet]$ returns the element-wise maximum of the vectors $\mathbf{h}_m$ that are neighbors to the current node n.

13.8.6 Aggregation by attention

The aggregation methods discussed so far either weight the contribution of the neighbors equally or in a way that depends on the graph topology. Conversely, in *graph attention layers*, the weights depend on the data at the nodes. A linear transform is applied to the current node embeddings so that:

$$\mathbf{H}'_k = \boldsymbol{\beta}_k \mathbf{1}^T + \boldsymbol{\Omega}_k \mathbf{H}_k. \tag{13.22}$$

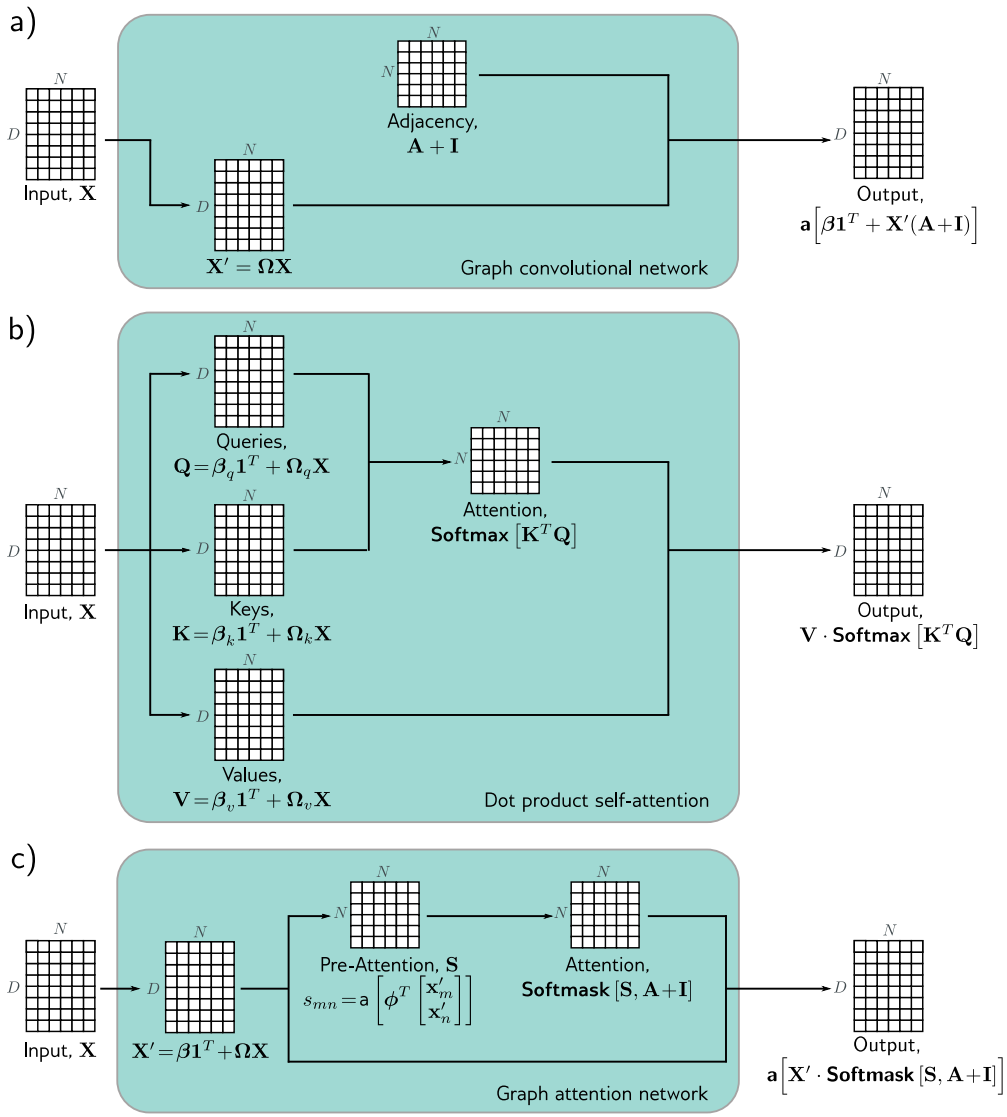

Figure 13.12 Comparison of graph convolutional network, dot product attention, and graph attention network. In each case, the mechanism maps N embeddings of size D stored in a $D \times N$ matrix $\mathbf{X}$ to an output of the same size. a) The graph convolutional network applies a linear transformation $\mathbf{X}' = \boldsymbol{\Omega}\mathbf{X}$ to the data matrix. It then computes a weighted sum of the transformed data, where the weighting is based on the adjacency matrix. A bias $\boldsymbol{\beta}$ is added, and the result is passed through an activation function. b) The outputs of the dot-product self-attention mechanism in the transformer are also weighted sums of the transformed inputs, but this time the weights depend on the data itself via the attention matrix. c) The graph attention network combines both of these mechanisms; the weights are both computed from the data and based on the adjacency matrix.

Then the similarity s_{mn} of each transformed node embedding $\mathbf{h}'_m$ to the transformed node embedding $\mathbf{h}'_n$ is computed by concatenating the pairs, taking a dot product with a column vector $\boldsymbol{\phi}_k$ of learned parameters, and applying an activation function:

$$s_{mn} = \mathrm{a}\left[\boldsymbol{\phi}_k^T \begin{bmatrix}\mathbf{h}'_m \\ \mathbf{h}'_n\end{bmatrix}\right]. \tag{13.23}$$

These variables are stored in an $N \times N$ matrix $\mathbf{S}$, where each element represents the similarity of every node to every other. As in dot-product self-attention, the attention weights contributing to each output embedding are normalized to be positive and sum to one using the softmax operation. However, only those values corresponding to the current node and its neighbors should contribute. The attention weights are applied to the transformed embeddings:

$$\mathbf{H}_{k+1} = \mathbf{a}\Big[\mathbf{H}'_k \cdot \mathbf{Softmask}[\mathbf{S}, \mathbf{A}+\mathbf{I}]\Big], \tag{13.24}$$

where $\mathbf{a}[\bullet]$ is a second activation function. The function $\mathbf{Softmask}[\bullet, \bullet]$ computes the attention values by applying softmax operation separately to each column of its first argument $\mathbf{S}$, but only after setting values where the second argument $\mathbf{A}+\mathbf{I}$ is zero to negative infinity, so they do not contribute. This ensures that the attention to non-neighboring nodes is zero.

Notebook 13.4 Graph attention

This is very similar to the dot-product self-attention computation in transformers (see figure 13.12), except that (i) The keys, queries, and values are all the same, (ii) The measure of similarity is different, and (iii) The attentions are masked so that each node only attends to itself and its neighbors. As in transformers, this system can be extended to use multiple heads that are run in parallel and recombined.

Problem 13.10

13.9 Edge graphs

Until now, we have focused on processing node embeddings. These evolve as they are passed through the network so that by the end of the network, they represent both the node and its context in the graph. We now consider the case where the information is associated with the edges of the graph.

It is easy to adapt the machinery for node embeddings to process edge embeddings using the *edge graph* (also known as the *adjoint graph* or *line graph*). This is a complementary graph, in which each edge in the original graph becomes a node, and every two edges with a common node in the original graph create an edge in the new graph (figure 13.13). In general, a graph can be recovered from its edge graph, so it's possible to swap between these two representations.

Problems 13.11–13.13

To process edge embeddings, the graph is translated to its edge graph. Then we use exactly the same techniques, aggregating information at each new node from its neighbors and combining this with the current representation. When both node and edge embeddings are present, we can translate back and forth between the two graphs. Now there are four possible updates (nodes update nodes, nodes update edges, edges update nodes, and edges update edges), and these can be alternated as desired, or with

Problem 13.14

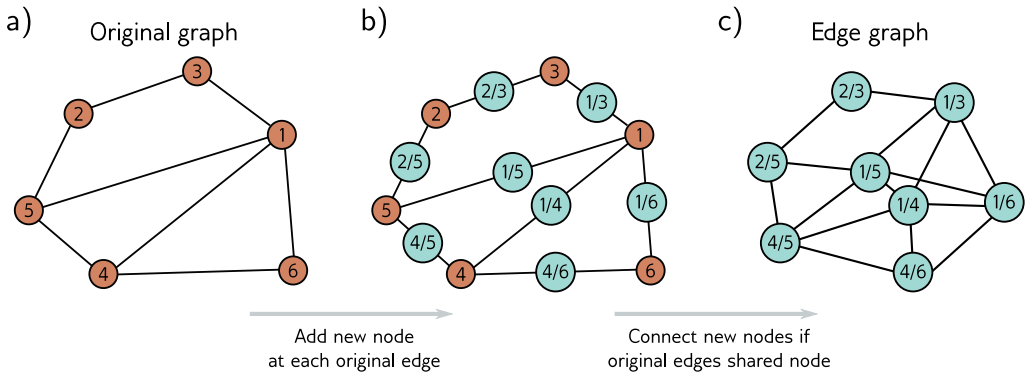

Figure 13.13 Edge graph. a) Graph with six nodes. b) To create the edge graph, we assign one node for each original edge (cyan circles), and c) connect the new nodes if the edges they represent connect to the same node in the original graph.

minor modifications, nodes can be updated simultaneously from both nodes and edges.

13.10 Summary

Graphs consist of a set of nodes, where pairs of these nodes are connected by edges. Both nodes and edges can have data attached, and these are referred to as node embeddings and edge embeddings, respectively. Many real-world problems can be framed in terms of graphs, where the goal is to establish a property of the entire graph, properties of each node or edge, or the presence of additional edges in the graph.

Graph neural networks are deep learning models that are applied to graphs. Since the node order in graphs is arbitrary, the layers of graph neural networks must be equivariant to permutations of the node indices. Spatial-based convolutional networks are a family of graph neural networks that aggregate information from the neighbors of a node and then use this to update the node embeddings.

One challenge of processing graphs is that they often occur in the transductive setting, where there is only one partially labeled graph rather than sets of training and test graphs. This graph can be extremely large, which adds further challenges in terms of training and has led to sampling and partitioning algorithms. The edge graph has a node for every edge in the original graph. By converting to this representation, graph neural networks can be used to update the edge embeddings.

Notes

Sanchez-Lengeling et al. (2021) and Daigavane et al. (2021) present good introductory articles on graph processing using neural networks. Recent surveys of research in graph neural networks can be found in articles by Zhou et al. (2020a), Wu et al. (2020c), and Veličković (2023), and the books of Hamilton (2020) and Ma & Tang (2021). GraphEDM (Chami et al., 2020) unifies many existing graph algorithms into a single framework. In this chapter, we have related graphs to convolutional networks following Bruna et al. (2013), but there are also strong connections with belief propagation (Dai et al., 2016) and graph isomorphism tests (Hamilton et al., 2017a). Zhang et al. (2019c) provide a review focusing specifically on graph convolutional networks. Bronstein et al. (2021) provide a general overview of geometric deep learning, including learning on graphs. Loukas (2020) discusses what types of functions graph neural networks can learn.

Applications: Applications include graph classification (e.g., Zhang et al., 2018b), node classification (e.g., Kipf & Welling, 2017), edge prediction (e.g., Zhang & Chen, 2018), graph clustering (e.g., Tsitsulin et al., 2020), and recommender systems (e.g., Wu et al., 2023). Methods for node classification are reviewed by Xiao et al. (2022a), methods for graph classification by Errica et al. (2019), and methods for edge prediction by Mutlu et al. (2020) and Kumar et al. (2020a).

Graph neural networks: Graph neural networks were introduced by Gori et al. (2005) and Scarselli et al. (2008), who formulated them as a generalization of recursive neural networks. The latter model used the iterative update:

$$\mathbf{h}_n \leftarrow \mathbf{f}\left[\mathbf{x}_n, \mathbf{x}_{m\in \mathrm{ne}[n]}, \mathbf{e}_{e\in \mathrm{nee}[n]}, \mathbf{h}_{m\in \mathrm{ne}[n]}, \boldsymbol{\phi}\right], \tag{13.25}$$

in which each node embedding $\mathbf{h}_n$ is updated from the initial embedding $\mathbf{x}_n$, initial embeddings $\mathbf{x}_{m\in \mathrm{ne}[n]}$ at the adjacent nodes, initial embeddings $\mathbf{e}_{e\in \mathrm{nee}[n]}$ at the adjacent edges, and adjacent node embeddings $\mathbf{h}_{m\in \mathrm{ne}[n]}$. For convergence, the function $\mathbf{f}[\bullet, \bullet, \bullet, \bullet, \boldsymbol{\phi}]$ must be a contraction mapping (see figure 16.9). If we unroll this equation in time for K steps and allow different parameters $\boldsymbol{\phi}_k$ at each time K, then equation 13.25 becomes similar to the graph convolutional network. Subsequent work extended graph neural networks to use gated recurrent units (Li et al., 2016b) and long short-term memory networks (Selsam et al., 2019).

Spectral methods: Bruna et al. (2013) applied the convolution operation in the Fourier domain. The Fourier basis vectors can be found by taking the eigendecomposition of the *graph Laplacian matrix*, $\mathbf{L} = \mathbf{D} - \mathbf{A}$ where $\mathbf{D}$ is the degree matrix and $\mathbf{A}$ is the adjacency matrix. This has disadvantages: the filters are not localized, and the decomposition is prohibitively expensive for large graphs. Henaff et al. (2015) tackled the first problem by forcing the Fourier representation to be smooth (and hence the spatial domain to be localized). Defferrard et al. (2016) introduced ChebNet, which approximates the filters efficiently by using the recursive properties of Chebyshev polynomials. This both provides spatially localized filters and reduces the computation. Kipf & Welling (2017) simplified this further to construct filters that use only a 1-hop neighborhood, resulting in a formulation similar to the spatial methods described in this chapter and providing a bridge between spectral and spatial methods.

Spatial methods: Spectral methods are ultimately based on the Graph Laplacian, so if the graph changes, the model must be retrained. This problem spurred the development of spatial methods. Duvenaud et al. (2015) defined convolutions in the spatial domain, using a different weight matrix to combine the adjacent embeddings for each node degree. This has the disadvantage that it becomes impractical if some nodes have a very large number of connections. Diffusion convolutional neural networks (Atwood & Towsley, 2016) use powers of the normalized adjacency matrix to blend features across different scales, sum these, pointwise multiply by

weights, and pass through an activation function to create the node embeddings. Gilmer et al. (2017) introduced *message-passing neural networks*, which defined convolutions on the graph as propagating messages from spatial neighbors. The "aggregate and combine" formulation of *GraphSAGE* (Hamilton et al., 2017a) fits into this framework.

Aggregate and combine: *Graph convolutional networks* (Kipf & Welling, 2017) take a weighted average of the neighbors and current node and then apply a linear mapping and ReLU. *GraphSAGE* (Hamilton et al., 2017a) applies a neural network layer to each neighbor, taking the elementwise maximum to aggregate. Chiang et al. (2019) propose *diagonal enhancement* in which the previous embedding is weighted more than the neighbors. Kipf & Welling (2017) introduced Kipf normalization, which normalizes the sum of the neighboring embeddings based on the degrees of the current node and its neighbors (see equation 13.19).

The *mixture model network* or *MoNet* (Monti et al., 2017) takes this one step further by *learning* a weighting based on the degrees of the current node and the neighbor. They associate a pseudo-coordinate system with each node, where the positions of the neighbors depend on these two quantities. They then learn a continuous function based on a mixture of Gaussians and sample this at the pseudo-coordinates of the neighbors to get the weights. In this way, they can learn the weightings for nodes and neighbors with arbitrary degrees. Pham et al. (2017) use a linear interpolation of the node embedding and neighbors with a different weighted combination for each dimension. The weight of this gating mechanism is generated as a function of the data.

Higher-order convolutional layers: Zhou & Li (2017) used higher-order convolutions by replacing the adjacency matrix $\mathbf{A}$ with $\tilde{\mathbf{A}} = \mathbf{Min}[\mathbf{A}^L + \mathbf{I}, \mathbf{1}]$ where L is the maximum walk-length, $\mathbf{1}$ is a matrix containing only ones, and $\mathbf{Min}[\bullet]$ takes the pointwise minimum of its two matrix arguments; the updates now sum together contributions from any nodes where there is at least one walk of length L. Abu-El-Haija et al. (2019) proposed *MixHop*, which computes node updates from the neighbors (using the adjacency matrix $\mathbf{A}$), the neighbors of the neighbors (using $\mathbf{A}^2$), and so on. They concatenate these updates at each layer. Lee et al. (2018) combined information from nodes beyond the immediate neighbors using geometric *motifs*, which are small local geometric patterns in the graph (e.g., a fully connected clique of five nodes).

Residual connections: Kipf & Welling (2017) proposed a residual connection in which the original embeddings are added to the updated ones. Hamilton et al. (2017b) concatenate the previous embedding to the output of the next layer (see equation 13.16). Rossi et al. (2020) present an inception-style network where the node embedding is concatenated to not only the aggregation of its neighbors but also the aggregation of all neighbors within a walk of two (via computing powers of the adjacency matrix). Xu et al. (2018) introduced *jump knowledge connections* in which the final output at each node consists of the concatenated node embeddings throughout the network. Zhang & Meng (2019) present a general formulation of residual embeddings called *GResNet* and investigate several variations in which the embeddings from the previous layer are added, the input embeddings are added, or versions of these that aggregate information from their neighbors (without further transformation) are added.

Attention in graph neural networks: Veličković et al. (2019) developed the *graph attention network* (figure 13.12c). Their formulation uses multiple heads whose outputs are combined symmetrically. *Gated Attention Networks* (Zhang et al., 2018a) weight the output of the different heads in a way that depends on the data itself. *Graph-BERT* (Zhang et al., 2020) performs node classification using self-attention alone; the graph's structure is captured by adding position embeddings to the data, similarly to how the absolute or relative position of words is captured in the transformer (chapter 12). For example, they add positional information that depends on the number of hops between nodes in the graph.

Permutation invariance: In *DeepSets*, Zaheer et al. (2017) presented a general permutation invariant operator for processing sets. Janossy pooling (Murphy et al., 2018) accepts that many functions are not permutation equivariant and instead uses a permutation-sensitive function and averages the results across many permutations.

Edge graphs: The notation of the *edge graph*, *line graph*, or *adjoint graph* dates to Whitney (1932). The idea of "weaving" layers that update node embeddings from node embeddings, node embeddings from edge embeddings, edge embeddings from edge embeddings, and edge embeddings from node embeddings was proposed by Kearnes et al. (2016). However, here the node-node and edge-edge updates do not involve the neighbors. Monti et al. (2018) introduced the *dual-primal graph CNN*, a modern formulation in a CNN framework that alternates between updates in the original and edge graphs.

Power of graph neural networks: Xu et al. (2019) argue that a neural network should be able to distinguish different graph structures; it is undesirable to map two graphs to the same output if they have the same initial node embeddings but different adjacency matrices. They identified graph structures that could not be distinguished by previous approaches such as GCNs (Kipf & Welling, 2017) and GraphSAGE (Hamilton et al., 2017a). They developed a more powerful architecture with the same discriminative power as the Weisfeiler-Lehman graph isomorphism test (Weisfeiler & Leman, 1968), which is known to discriminate a broad class of graphs. This resulting *graph isomorphism network* was based on the aggregation operation:

$$\mathbf{h}_{k+1}^{(n)} = \mathbf{mlp}\left[(1+\epsilon_k)\,\mathbf{h}_k^{(n)} + \sum_{m\in \mathrm{ne}[n]} \mathbf{h}_k^{(m)}\right]. \tag{13.26}$$

Batches: The original paper on graph convolutional networks (Kipf & Welling, 2017) used full-batch gradient descent. This has memory requirements proportional to the number of nodes, embedding size, and number of layers during training. Since then, three types of methods have been proposed to reduce the memory requirements and create batches for SGD in the transductive setting: node sampling, layer sampling, and sub-graph sampling.

Node sampling methods start by randomly selecting a subset of target nodes and then work back through the network, adding a subset of the nodes in the receptive field at each stage. GraphSAGE (Hamilton et al., 2017a) proposed a fixed number of neighborhood samples as in figure 13.10b. Chen et al. (2018b) introduce a variance reduction technique, but this uses historical activations of nodes and so still has a high memory requirement. *PinSAGE* (Ying et al., 2018a) uses random walks from the target nodes and chooses the K nodes with the highest visit count. This prioritizes ancestors that are more closely connected.

Node sampling still requires increasing numbers of nodes as we pass back through the graph. *Layer sampling methods* address this by directly sampling the receptive field in each layer independently. Examples of layer sampling include FastGCN (Chen et al., 2018a), adaptive sampling (Huang et al., 2018b), and layer-dependent importance sampling (Zou et al., 2019).

Subgraph sampling methods randomly draw subgraphs or divide the original graph into subgraphs. These are then trained as independent data examples. Examples of these approaches include *GraphSAINT* (Zeng et al., 2020), which samples sub-graphs during training using random walks and then runs a full GCN on the subgraph while also correcting for the bias and variance of the minibatch. *Cluster GCN* (Chiang et al., 2019) partitions the graph into clusters (by maximizing the embedding utilization or number of within-batch edges) in a pre-processing stage and randomly selects clusters to form minibatches. To create more randomness, they train random subsets of these clusters plus the edges between them (see figure 13.11).

Wolfe et al. (2021) proposed a distributed training method that both partitions the graph and trains narrower GCNs in parallel by partitioning the feature space at different layers. More information about sampling graphs can be found in Rozemberczki et al. (2020).

Regularization and normalization: Rong et al. (2020) proposed *DropEdge*, which randomly drops edges from the graph during each training iteration by masking the adjacency matrix. This can be done for the whole neural network or differently in each layer (layer-wise DropEdge). In a sense, this is similar to dropout in that it breaks connections in the flow of data, but it can also be considered an augmentation method since changing the graph is similar to perturbing the data. Schlichtkrull et al. (2018), Teru et al. (2020), and Veličković et al. (2019) also proposed randomly dropping edges from the graph as a form of regularization similar to dropout. Node sampling methods (Hamilton et al., 2017a; Huang et al., 2018b; Chen et al., 2018a) can also be considered regularizers. Hasanzadeh et al. (2020) present a general framework called *DropConnect* that unifies many of the above approaches.

There are also many proposed normalization schemes for graph neural networks, including *PairNorm* (Zhao & Akoglu, 2020), *weight normalization* (Oono & Suzuki, 2019), *differentiable group normalization* (Zhou et al., 2020b), and *GraphNorm* (Cai et al., 2021).

Multi-relational graphs: Schlichtkrull et al. (2018) proposed a variation of graph convolutional networks for multi-relational graphs (i.e., graphs with more than one edge type). Their scheme separately aggregates information from each edge type using different parameters. If there are many edge types, the number of parameters may become large, and to combat this, they propose that each edge type uses a different weighting of a basis set of parameters.

Hierarchical representations and pooling: CNNs for image classification gradually decrease the representation size but increase the number of channels as the network progresses. However, the GCNs for graph classification in this chapter maintain the entire graph until the last layer and then combine all the nodes to compute the final prediction. Ying et al. (2018b) proposed *DiffPool*, which clusters graph nodes to make a graph that gets progressively smaller as the depth increases in a way that is differentiable, and so can be learned. This can be done based on the graph structure alone or adaptively based on the graph structure and the embeddings. Other pooling methods include *SortPool* (Zhang et al., 2018b) and *self-attention graph pooling* (Lee et al., 2019). A comparison of pooling layers for graph neural networks can be found in Grattarola et al. (2022). Gao & Ji (2019) propose an encoder-decoder structure for graphs based on the U-Net (see figure 11.10).

Geometric graphs: The *MoNet* model (Monti et al., 2017) can exploit geometric information because neighboring nodes have well-defined spatial positions. They learn a mixture of Gaussians function and sample from this based on the relative coordinates of the neighbor. In this way, they can weight neighboring nodes based on their relative positions as in standard convolutional neural networks, even though these positions are not constant. The *geodesic CNN* (Masci et al., 2015) and *anisotropic CNN* (Boscaini et al., 2016) both adapt convolution to manifolds (i.e., surfaces) as represented by triangular meshes. They locally approximate the surface as a plane and define a coordinate system on this plane around the current node.

Oversmoothing and suspended animation: Unlike other deep learning models, graph neural networks did not, until recently, benefit significantly from increasing depth. Indeed, the original GCN paper (Kipf & Welling, 2017) and GraphSAGE (Hamilton et al., 2017a) both only use two layers, and Chiang et al. (2019) trained a five-layer Cluster-GCN to get state-of-the-art performance on the PPI dataset. One possible explanation is *over-smoothing* (Li et al., 2018c); at each layer, the network incorporates information from a larger neighborhood, and it may be that this ultimately results in the dissolution of (important) local information. Indeed (Xu et al., 2018) prove that the influence of one node on another is proportional to the probability

of reaching that node in a K-step random walk. This approaches the stationary distribution of walks over the graph with increasing K, causing the local neighborhood to be washed out.

Alon & Yahav (2021) proposed another explanation for why performance doesn't improve with network depth. They argue that adding depth allows information to be aggregated from longer paths. However, in practice, the exponential growth in the number of neighbors means there is a bottleneck whereby too much information is "squashed" into the fixed-size node embeddings.

Ying et al. (2018a) also note that when the depth of the network exceeds a certain limit, the gradients no longer propagate back, and learning fails for both the training and test data. They term this effect *suspended animation*. This is similar to when many layers are naïvely added to convolutional neural networks (figure 11.2). They propose a family of residual connections that allow deeper networks to be trained. Vanishing gradients (section 7.5) have also been identified as a limitation by Li et al. (2021b).

It has recently become possible to train deeper graph neural networks using various forms of residual connection (Xu et al., 2018; Li et al., 2020a; Gong et al., 2020; Chen et al., 2020b; Xu et al., 2021a). Li et al. (2021a) train a state-of-the-art model with more than 1000 layers using an invertible network to reduce the memory requirements of training (see chapter 16).

Problems

Problem 13.1 Write out the adjacency matrices for the two graphs in figure 13.14.

Problem 13.2* Draw graphs that correspond to the following adjacency matrices:

$$\mathbf{A}_1 = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 \end{bmatrix} \quad \text{and} \quad \mathbf{A}_2 = \begin{bmatrix} 0 & 0 & 1 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 1 & 1 & 0 \end{bmatrix}.$$

Problem 13.3* Consider the two graphs in figure 13.14. How many ways are there to walk from node one to node two in (i) three steps and (ii) seven steps?

Problem 13.4 The diagonal of $\mathbf{A}^2$ in figure 13.4c contains the number of edges that connect to each corresponding node. Explain this phenomenon.

Appendix B.4.4 Permutation matrix

Problem 13.5 What permutation matrix is responsible for the transformation between the graphs in figures 13.5a–c and figure 13.5d–f?

Problem 13.6 Prove that:

$$\text{sig}\left[\beta_K + \boldsymbol{\omega}_K \mathbf{H}_K \mathbf{1}\right] = \text{sig}\left[\beta_K + \boldsymbol{\omega}_K \mathbf{H}_K \mathbf{P} \mathbf{1}\right], \tag{13.27}$$

where $\mathbf{P}$ is an $N \times N$ permutation matrix (a matrix that is all zeros except for exactly one entry in each row and each column, which is one), and $\mathbf{1}$ is an $N \times 1$ vector of ones.

Problem 13.7* Consider the simple GNN layer:

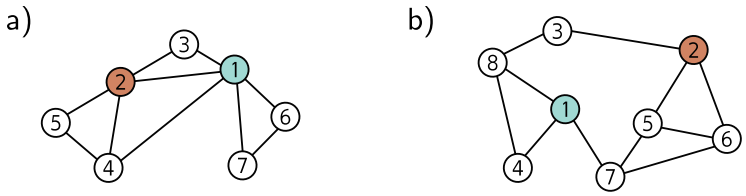

Figure 13.14 Graphs for problems 13.1, 13.3, and 13.8.

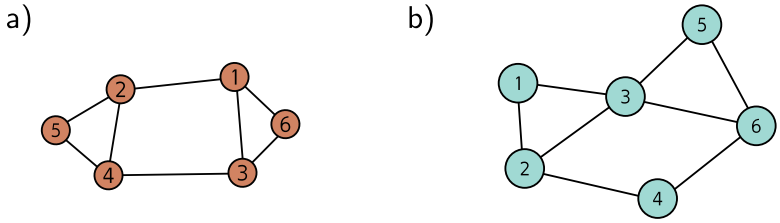

Figure 13.15 Graphs for problems 13.11–13.13.

$$\begin{aligned}\mathbf{H}_{k+1} &= \text{GraphLayer}[\mathbf{H}_k, \mathbf{A}] \\ &= \mathbf{a}\left[\boldsymbol{\beta}_k \mathbf{1}^T + \boldsymbol{\Omega}_k \begin{bmatrix}\mathbf{H}_k \\ \mathbf{H}_k\mathbf{A}\end{bmatrix}\right], \end{aligned} \tag{13.28}$$

where $\mathbf{H}$ is a $D \times N$ matrix containing the N node embeddings in its columns, $\mathbf{A}$ is the $N \times N$ adjacency matrix, $\boldsymbol{\beta}$ is the bias vector, and $\boldsymbol{\Omega}$ is the weight matrix. Show that this layer is equivariant to permutations of the node order so that:

$$\text{GraphLayer}[\mathbf{H}_k, \mathbf{A}]\mathbf{P} = \text{GraphLayer}[\mathbf{H}_k\mathbf{P}, \mathbf{P}^T\mathbf{A}\mathbf{P}], \tag{13.29}$$

where $\mathbf{P}$ is an $N \times N$ permutation matrix.

Problem 13.8 What is the degree matrix $\mathbf{D}$ for each graph in figure 13.14?

Problem 13.9 The authors of GraphSAGE (Hamilton et al., 2017a) propose a pooling method in which the node embedding is averaged together with its neighbors so that:

$$\mathbf{agg}[n] = \frac{1}{1+|\text{ne}[n]|}\left(\mathbf{h}_n + \sum_{m \in \text{ne}[n]} \mathbf{h}_m\right). \tag{13.30}$$

Show how this operation can be computed simultaneously for all node embeddings in the $D \times N$ embedding matrix $\mathbf{H}$ using linear algebra. You will need to use both the adjacency matrix $\mathbf{A}$ and the degree matrix $\mathbf{D}$.

Problem 13.10* Devise a graph attention mechanism based on dot-product self-attention and draw its mechanism in the style of figure 13.12.

Problem 13.11* Draw the edge graph associated with the graph in figure 13.15a.

Problem 13.12* Draw the node graph corresponding to the edge graph in figure 13.15b.

Problem 13.13 For a general undirected graph, describe how the adjacency matrix of the node graph relates to the adjacency matrix of the corresponding edge graph.

Problem 13.14* Design a layer that updates a node embedding $\mathbf{h}_n$ based on its neighboring node embeddings $\{\mathbf{h}_m\}_{m\in \text{ne}[n]}$ and neighboring edge embeddings $\{\mathbf{e}_m\}_{m\in \text{nee}[n]}$. You should consider the possibility that the edge embeddings are not the same size as the node embeddings.